3.1 Purpose of Constants

Constants are fixed values used in a program. Defining tariff rates and limits as constants allows changes to be made in one place, making the program easier to maintain.

3.2 Types of Constants in C

- "#define": Creates symbolic constants through the preprocessor. It has no data type.
- "const" variables: Have a data type and cannot be modified through that variable after initialization.
- "enum": Defines named integer constants, useful for categories and options.

3.3 Difference Between "#define" and "const"

"#define" performs text substitution before compilation and has no scope or type checking of its own. A "const" variable has a data type and scope, making it easier for the compiler and debugger to inspect.

3.4 Naming Conventions

Constants are commonly written in uppercase letters with underscores, such as "RATE1_PAISE", "RATE2_PAISE", and "FIXED_CHARGE_PAISE". Meaningful names improve readability and reduce errors.

3.5 Alternative Concept and Complexity Analysis

Using "const" variables instead of "#define" provides type checking and scope. Using "enum" is suitable for named integer values but not for monetary amounts. These choices generally do not significantly affect running time. A well-defined set of constants makes tariff revisions easier.

```
#include <stdio.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(unsigned int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB1_UNITS) {
        if (units <= SLAB2_UNITS)
            paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        else {
            paise += (long)(SLAB2_UNITS - SLAB1_UNITS)
                    * RATE2_PAISE;
            paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        }
    }

    return paise;
}

int main(void)
{
    unsigned int tests[] = {0, 100, 150, 200, 350};
```

```
    int n = sizeof(tests) / sizeof(tests[0]);

    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%u units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }

    return 0;
}
```

